ACESSANDO BANCO DE DADOS COM JAVA

NOSSA HISTÓRIA

A nossa história inicia com a realização do sonho de um grupo de empresários, em atender à crescente demanda de alunos para cursos de Graduação e Pós-Graduação. Com isso foi criado a nossa instituição, como entidade oferecendo serviços educacionais em nível superior.

A instituição tem por objetivo formar diplomados nas diferentes áreas de conhecimento, aptos para a inserção em setores profissionais e para a participação no desenvolvimento da sociedade brasileira, e colaborar na sua formação contínua. Além de promover a divulgação de conhecimentos culturais, científicos e técnicos que constituem patrimônio da humanidade e comunicar o saber através do ensino, de publicação ou outras normas de comunicação.

A nossa missão é oferecer qualidade em conhecimento e cultura de forma confiável e eficiente para que o aluno tenha oportunidade de construir uma base profissional e ética. Dessa forma, conquistando o espaço de uma das instituições modelo no país na oferta de cursos, primando sempre pela inovação tecnológica, excelência no atendimento e valor do serviço oferecido.

Sumário

INTRODUÇÃO 4

O BANCO DE DADOS JAVA 5

CONFIGURANDO O BANCO DE DADOS 8

REGISTRANDO O BANCO DE DADOS 10

INICIANDO UM SERVIDOR E CRIANDO O BANCO DE DADOS 11

O QUE PRECISAMOS PARA ACESSAR O BANCO DE DADOS COM JAVA 13

O ACESSO AO BANCO DE DADOS UTILIZANDO JAVA 14

CONSULTA DE DADOS 18

MANIPULAÇÃO DE DADOS 20

ATUALIZAÇÕES 22

ENCERRANDO AS CONEXÕES 23

CONSIDERAÇÕES FINAIS 24

REFERÊNCIAS 25

INTRODUÇÃO

Sem dúvida alguma o acesso a dados é um dos recursos mais usados na área de desenvolvimento hoje em dia. Praticamente todo o sistema precisa de um banco de dados, havendo raras exceções. Neste artigo estaremos abordando o básico sobre esse assunto no Turbo JBuilder 2007, a IDE CodeGear para desenvolvimento Java, uma das linguagens mais faladas no mundo.

A linguagem de programação Java, diferente de linguagens como PHP, não suporta o acesso a banco de dados diretamente, para isso ele usa uma API (conjunto de classes e interfaces) para fazer o serviço. A JDBC (Java Database Connectivity), faz o envio de instruções SQL para qualquer banco de dados relacional, desde que haja um driver que corresponda ao mesmo presente.

Quando criamos um sistema, é muito comum termos alguma forma de persistir dados, ou seja, guardá-los em algum lugar para, posteriormente, consultá-los. A forma mais comum de se fazer isso é por meio de Sistemas Gerenciadores de Banco de Dados (SGBD). O Java permite, não só acesso a eles, mas, também, a uma variedade de fornecedores e produtos como: MySQL, Oracle, SQL Server (Microsoft), PostgreSQL, entre outros. Neste artigo, veremos como o JDBC torna isso é possível, bem como uma “receita de bolo” para usarmos um banco de dados em nossos programas.

O BANCO DE DADOS JAVA

O banco de dados Java DB é uma distribuição da Sun com suporte do Apache Derby. O Java DB é um servidor de banco de dados com base em padrões, seguro e totalmente transacional, escrito inteiramente em Java, e dá suporte total a SQL, JDBC API e à tecnologia Java EE. O banco de dados Java DB é encapsulado com o servidor de aplicações GlassFish e está incluído no JDK 6 também. Para obter mais informações sobre o banco de dados Java DB, consulte a documentação oficial.

O Java DB é instalado quando o JDK 7 ou JDK 8 é instalado (com exceção do Mac OS X). Se estiver usando Mac OS X é possível fazer o download e instalar o Java DB manualmente ou usar o Java DB que é instalado pela versão do Java EE do instalador do NetBeans IDE.

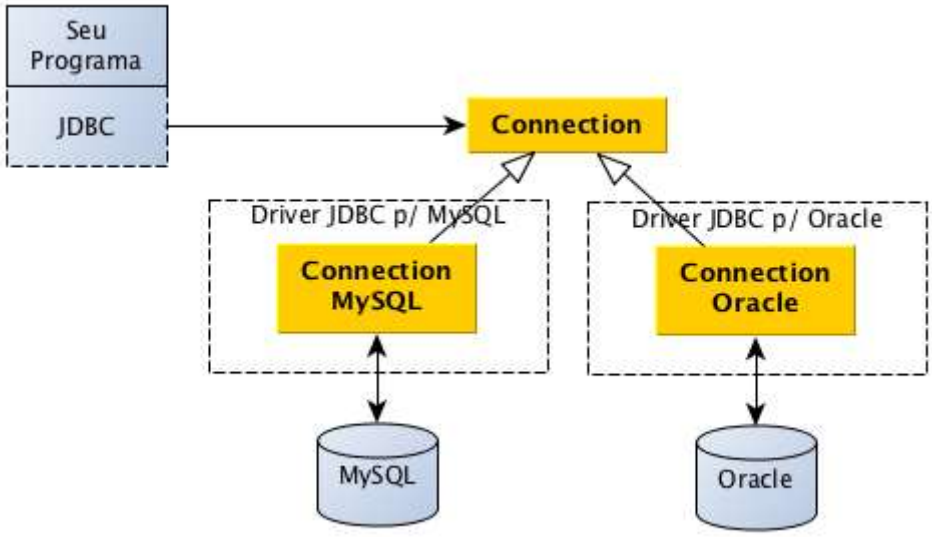
A API JDBC provê um conjunto de interfaces que permite um acesso uniforme a banco de dados em Java, deixando a cargo de cada fabricante implementar o acesso ao seu banco de dados da maneira que achar mais conveniente. Essas interfaces estão no pacote java.sql.*. Dessa forma, a primeira coisa que precisamos é baixar o banco de dados do banco de dados que você utilizará.

Quando queremos acessar um banco de dados, é necessário se conectar e, depois, se comunicar com ele. Essa comunicação é feita por meio de um protocolo específico para cada produto e/ou fornecedor. Assim sendo, é muito comum termos alguma biblioteca de funções ou classes para fazer esse trabalho pesado. Normalmente, o próprio fabricante do banco de dados fornece essa biblioteca sob o nome de “driver de acesso”. E, seu programa “conversa” **diretamente** com esse “driver”. Isso significa que, se você precisar trocar o acesso do seu programa para outra “marca” de banco de dados, precisará de outro “driver” e, provavelmente, terá que alterar seu código porque a utilização dessa nova biblioteca será diferente.

(Fonte: https://www.ramon.pro.br/jdbc-acessando-bancos-de-dados-em-java/)

O Java possui uma API (Application Programming Interface) para acesso a banco de dados chamada JDBC (Java DataBase Connectivity). Essa API permite usar os “drivers de acesso” de forma transparente, isto é, independentemente da “marca” do banco de dados. Com isso, se sua aplicação fizer uso apenas de comandos SQL básicos, é bem provável que ela fique extremamente flexível em relação ao banco de dados que você for usar.

Mas, como isso é possível? Bom, isso é possível pelo bom uso dos conceitos de Programação Orientada e Objetos e mais alguns truquezinhos “avançados” (como Reflection, Service Providers, entre outros). No fim, a “mágica” toda fica por conta do polimorfismo. Vejamos como, observando a figura abaixo:



(Fonte: https://www.ramon.pro.br/jdbc-acessando-bancos-de-dados-em-java/)

Note que o seu programa continua usando um driver, mas a “conversa” é feita indiretamente, por meio do JDBC e sua interface chamada `Connection`. Cada “Driver JDBC” implementa essa interface de forma específica para cada banco de dados. Usando os conceitos de Orientação a Objetos, seu programa “sabe” apenas da interface `Connection`, ficando livre de acessar classes e/ou funções específicas, ou seja, o código fica muito flexível. Observação: obviamente, o JDBC é composto por “muitas” outras interfaces e classes… `Connection` é apenas uma delas!

Mas, como é instanciado o objeto correto? Bom, o JDBC possui um classe chamada `DriverManager` que sabe ler uma configuração chamada “URL de conexão” (ou “string de conexão”) e, por meio dela, cria o objeto específico para o banco que se quer usar. Isso ficará mais claro logo abaixo, quando você vir o código!

CONFIGURANDO O BANCO DE DADOS

Se você tem o GlassFish Server Enterprise registrado na instalação NetBeans IDE, o Java DB já estará registrado. Portanto, é possível pular para Iniciando o Servidor e Criando um Banco de Dados.

Se fizer download do GlassFish Server separadamente e necessitar de ajuda ao registrar no NetBeans IDE, consulte Registrando uma Instância do GlassFish Server no Conteúdo de Ajuda IDE (F1).

Se você tiver submetido o Java DB a download, siga as etapas a seguir.

Execute o arquivo de autoextração. Uma pasta denominada 'javadb' será criada no mesmo local do arquivo. Se você tiver apenas submetido o Java DB a download e desejar que o servidor de banco de dados fique em um local diferente de onde ele foi extraído, deverá realocá-lo agora.

No sistema, crie um novo diretório para ser utilizado como diretório home para as instâncias individuais do servidor de banco de dados. Por exemplo, é possível criar esta pasta no diretório-raiz Java DB (javadb) ou em qualquer outra localização.

Antes de prosseguir, é importante compreender os componentes encontrados no diretório raiz do Java DB:

O subdiretório `demo` contém os programas de demonstração.

O subdiretório `bin` contém os scripts para executar utilitários e configurar o ambiente.

O subdiretório `javadoc` contém a documentação da API que foi gerada de comentários de código-fonte.

O subdiretório `docs` contém a documentação do Java DB.

O subdiretório `lib` contém os arquivos jar do Java DB.

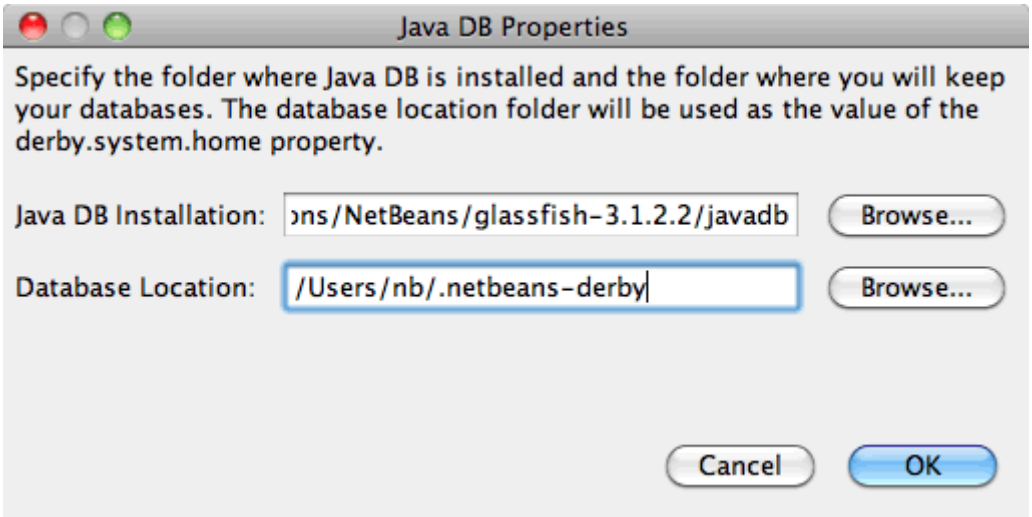
REGISTRANDO O BANCO DE DADOS

Agora que o banco de dados está configurado, execute as etapas a seguir para registrar o Java DB no IDE.

1. Na janela Serviços, clique com o botão direito do mouse no nó do Banco de Dados Java DB e escolha Propriedades para abrir a caixa de diálogo das Definições do Java DB.
2. No campo de texto Instalação de Java DB, insira o caminho para o diretório-raiz de Java DB (javadb) especificado na etapa anterior.
3. Para a Localização do Banco de Dados, use o local default se um local já foi fornecido. Clique em OK
4.

Por exemplo, a localização default pode parecer com `C:\Documents and Settings\username\.netbeans-derby` em um computador Windows.

Exemplo de definições default do servidor e banco de dados Java DB:



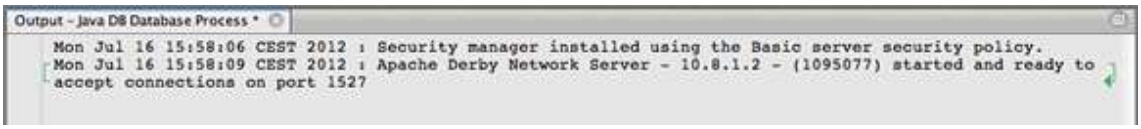
(Fonte: https://netbeans.apache.org//kb/docs/ide/java-db_pt_BR.html)

Observação: Se o campo Localização do Banco de Dados estiver vazio será necessário definir o caminho para o diretório que contém seus bancos de dados. Será necessário criar um diretório para os bancos de dados se nenhum diretório existir.

INICIANDO UM SERVIDOR E CRIANDO O BANCO DE DADOS

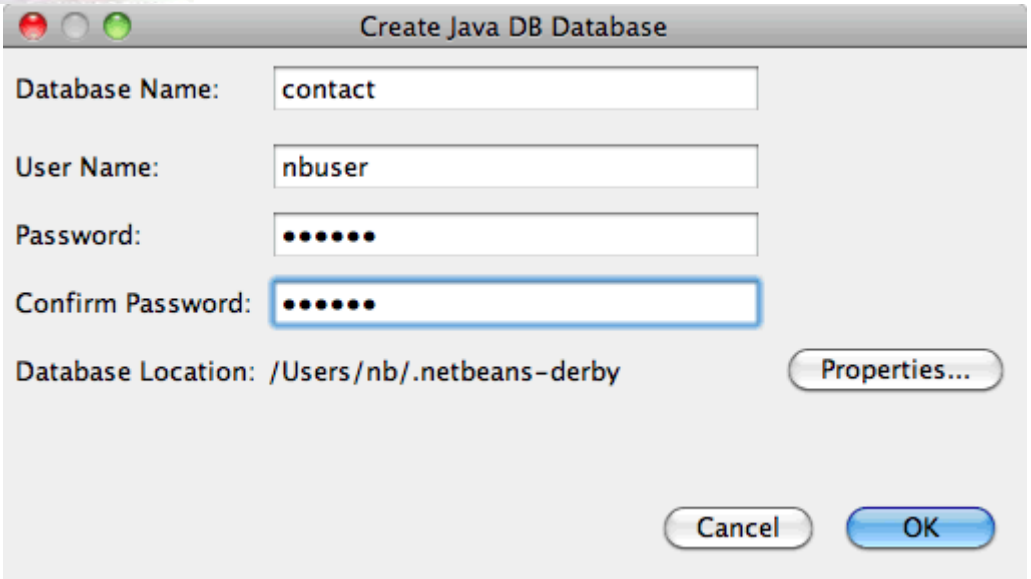
As opções de menu do Banco de Dados Java DB são exibidas quando você clica com o botão direito do mouse no nó Java DB na janela Serviços. Os itens desse menu contextual permitem o início e a interrupção do servidor de banco de dados, a criação de uma nova instância de banco de dados e o registro de servidores de bancos de dados no IDE (como demonstrado na etapa anterior). Para iniciar o servidor de banco de dados:

1. Na janela Serviços, clique com o botão direito do mouse no nó Java DB e escolha Iniciar Servidor. Observe a seguinte saída na janela de Saída, indicando que o servidor foi iniciado:



(Fonte: https://netbeans.apache.org//kb/docs/ide/java-db_pt_BR.html)

1. Clique com o botão direito do mouse no nó do Java DB e escolha Criar Banco de Dados para abrir a caixa de diálogo de criação do Banco de Dados do Java DB.
2. Insira contato para o Nome do Banco de Dados.
3. Digite nbuser para o Nome de Usuário e a Senha. Clique em OK.



(Fonte: https://netbeans.apache.org//kb/docs/ide/java-db_pt_BR.html)

Observação: Localização do Banco de Dados é o local default definido durante a instalação do Java DB do GlassFish. Se você tiver instalado o Java DB separadamente, essa localização pode ser diferente.

Após você criar o Banco de Dados, se você expandir os bancos de dados nó na janela Serviços que você pode ver o IDE criada uma conexão de banco de dados e que o banco de dados foi adicionado à lista sob o nó do BD Java.

O QUE PRECISAMOS PARA ACESSAR O BANCO DE DADOS COM JAVA

Para começar, saiba que precisamos, basicamente, de quatro informações para acessar um banco de dados:

- Nome completo (FQN) da classe que implementa o Driver JDBC.
- URL de conexão.
- Nome do usuário de acesso ao banco.
- Senha do usuário de acesso ao banco.

Para saber os dois primeiros itens, é necessário consultar o manual do Driver JDBC específico para o banco que você quer acessar. Ah! Lógico, você também irá precisar fazer o *download* do driver e colocá-lo junto ao seu programa (cada IDE faz de um jeito, mas, basicamente, a JVM irá acessa-lo por meio do CLASSPATH).

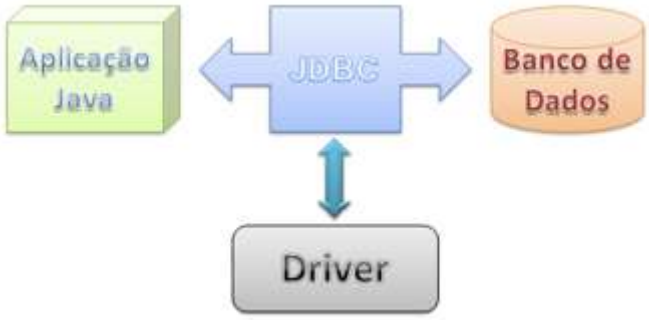
Por exemplo, se formos acessar um banco de dados MySQL, precisaremos fazer o download do Driver JDBC para MySQL e, olhando a documentação, saberemos que o nome completo da classe que implementa o Driver JDBC é "`com.mysql.jdbc.Driver`" e que a URL de conexão tem o seguinte formato: "`jdbc:mysql://host:port/database`". Observação: para os exemplos deste artigo, estarei usando um banco de dados de exemplo do MySQL chamado "sakila".

O ACESSO AO BANCO DE DADOS UTILIZANDO JAVA

O banco de dados é onde persistimos (armazenamos) os dados que pertencem ao nosso sistema. A maioria dos bancos de dados comerciais hoje em dia é do tipo relacional e derivam de uma estrutura diferente da orientada a objetos.

Para executarmos o manuseio de informações em um banco de dados, devemos fazer uso de sub-linguagens de banco de dados (Database Sub Language – DSL), voltadas para as operações do banco de dados. Geralmente, as sub-linguagens de dados são compostas da combinação de recursos para a definição de dados (Data Definition Language – DDL) e recursos específicos para a manipulação de dados (Data Manipulation Language – DML). A conhecida linguagem de consulta SQL (Structured Query Language) é uma destas linguagens que fornece suporte tanto a DDL como a DML.

Para efetivarmos a conexão de um programa desenvolvido em Java com uma base de dados qualquer, a linguagem Java implementa um conceito de ponte, que implementa todas as funcionalidades que um banco de dados padrão deve nos fornecer.



(Fonte: http://www.universidadejava.com.br/java/java-jdbc/)

No desenho acima podemos notar o relacionamento direto da JDBC com o banco de dados, porém este relacionamento depende também da extensão desta ponte que é representada pela implementação JDBC escolhida. Esta implementação depende do banco de dados com o qual queremos nos comunicar e a ela damos o nome de Driver. Para gerenciar estes drivers de conexão, a

linguagem Java possui um gerente de drivers chamado `java.sql.DriverManager`.

O driver do MySQL pode ser adicionado através das Propriedades do Projeto, na aba Biblioteca utilize o botão Adicionar Biblioteca e selecione MySQL JDBC Driver.

Para criarmos uma conexão com o banco de dados primeiramente é necessário informarmos qual driver de conexão o **DriverManager** deve utilizar.

Feita essa disponibilização do arquivo de driver, na programação se faz necessário registrar o driver do banco de dados no sistema. Para isso, basta carregá-lo através do método `Class.forName()`. Esse método abre uma classe que se registra com o `DriverManager.getConnection()`.

A partir da classe `DriverManager`, podemos utilizar um método chamado `getConnection`, com o qual poderemos nos conectar a uma determinada base de dados utilizando uma String padrão de conexão.

```
package material.jdbc;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * Classe utilizada para conectar e desconectar do banco de dados.
 */
public class Conexao {
  public static void main(String[] args) {
    Conexao conexao = new Conexao();
    Connection conn = conexao.conectar();
    conexao.desconectar(conn);
  }

  public Connection conectar() {
    Connection conn = null;
    try {
      Class.forName("com.mysql.jdbc.Driver");
      conn = DriverManager.getConnection("jdbc:mysql://loca-
lhost/test", "root", "root");
      System.out.println("Conectou no banco de dados.");
    } catch (SQLException ex) {
      System.out.println("Erro: Não conseguiu conectar no BD.");
    } catch (ClassNotFoundException ex) {
```

```
        System.out.println("Erro: Não encontrou o driver do BD.");
      }

      return conn;
    }

    public void desconectar(Connection conn) {
      try {
        if(conn != null && !conn.isClosed()) {
          conn.close();
          System.out.println("Desconectou do banco de dados.");
        }
      } catch (SQLException ex) {
        System.out.println("Não conseguiu desconectar do BD.");
      }
    }
  }
```

Na linha 20 estamos carregando o driver do banco de dados MySQL `com.mysql.jdbc.Driver`.

Na linha 21, utilizamos o método getConnection da classe `DriverManager` para criar uma conexão com o banco de dados armazenado em um objeto do tipo `java.sql.Connection`.

O método `getConnection()` recebe três parâmetros: 1˚ url de conexão com o banco de dados, 2˚ usuário e 3˚ senha.

Para fazer a conexão com o banco de dados precisamos tratar algumas exceções que podem ocorrer:

Na linha 23, tratamos a exceção `java.sql.SQLException` que é lançada caso não consiga criar uma conexão com o Banco de Dados.

Na linha 25, tratamos a exceção `java.lang.ClassNotFoundException` que é lançada caso não consiga carregar o driver do banco de dados.

Depois de utilizar a conexão com o banco de dados é muito importante que liberemos esta conexão, ou seja, precisamos encerrar a conexão com o banco de dados.

Na linha 32 temos o método desconectar que recebe uma Connection como parâmetro, dentro deste método estamos verificando se a conexão com o banco ainda não foi encerrada para depois encerra-la. Quando tentamos encerrar uma conexão com o banco também pode ocorrer alguma exceção para isso na linha 38 estamos tratando caso ocorra alguma `java.sql.SQLException`.

Quando executamos esta classe, temos a seguinte saída no console:

```
Conectou no banco de dados.
Desconectou do banco de dados.
```

CONSULTA DE DADOS

Uma vez conectado, podemos então solicitar a execução de comandos SQL, para que tenhamos acesso a este tipo de chamada ao banco de dados é necessário criarmos um `java.sql.Statement`.

O `Statement` é o objeto que servirá de instrumento para a execução de comandos SQL, porém é importante ressaltar que o intuito de uma consulta a um banco de dados é o de receber uma informação, logo, é fundamental que essa resposta seja armazenada em uma estrutura. Para isso podemos utilizar a classe `java.sql.ResultSet` que deve receber o retorno do método `executeQuery` que é responsável por executar a consulta.

Exemplo:

```
package material.jdbc;

import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

/**
 * Classe utilizada para demonstrar a consulta ao banco de dados.
 */
public class Consulta {
  public static void main (String[] args) {
    Consulta consultaBD = new Consulta();
    consultaBD.consulta();
  }

  public void consulta() {
    Conexao conexao = new Conexao();
    Connection conn = conexao.conectar();
    try {
      String consulta = "SELECT * FROM PESSOA WHERE NOME like 'A%'";

      Statement stm = conn.createStatement();
      ResultSet resultado = stm.executeQuery(consulta);

      while(resultado.next()) {
        System.out.print(resultado.getLong("ID"));
        System.out.print(" - " + resultado.getString("NOME"));
        System.out.print(" - " + resultado.getInt("IDADE") + "\n");
      }
    } catch (SQLException ex) {
      System.out.println("Não conseguiu consultar os dados de Pes-
soa.");
```

```
    } finally {
      conexao.desconectar(conn);
    }
  }
}
```

Quando executamos essas classe, temos a seguinte saída no console:

```
Conectou no banco de dados.
1 - Ana - 30
5 - Amanda - 23
Desconectou do banco de dados.
```

Note pelo exemplo anterior que o método responsável por executar a consulta na base de dados foi o `executeQuery` na linha 24. Observe também a forma como o `ResultSet` foi percorrido na linha 26.

A classe `ResultSet` oferece o método `next()` para percorrer a resposta dada pelo banco de dados. Caso nossa consulta acima lista houvesse retornado 4 linhas, por padrão o `ResultSet` estaria com seu ponteiro posicionado na posição `-1`, logo, antes de ler a primeira linha de resposta de sua consulta é necessário a chamada a este método `next()` para que o ponteiro passe para a primeira posição e possa resgatar as informações desejadas. O método `next()` retorna `true`, caso ainda existam novas linhas na resposta e `false` quando não tem mais registros para percorrer.

É importante ressaltar que apenas um único objeto `ResultSet` pode ser aberto por vez por `Statement` ao mesmo tempo.

Para resgatar as informações desejadas, a classe `ResultSet` oferece métodos do tipo `get` para todos os tipos de dados primitivos, exceto `char`. Todos estes métodos devem receber como parâmetro o nome da coluna de resposta, assim como no exemplo anterior, ou o número correspondente a posição da coluna retornada.

MANIPULAÇÃO DE DADOS

Podemos utilizar a classe Statement para guardar (persistir) ou atualizar as informações na base de dados, utilizando o método execute.

```
package material.jdbc;

import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;

/**
 * Classe utilizada para demonstrar como adicionar ou atualizar dados
 *  no banco de dados.
 */
public class Manipulacao {
  public static void main(String[] args) {
    Manipulacao manipulacao = new Manipulacao();
    manipulacao.inserir();
    manipulacao.atualizar();
  }

  public void inserir() {
    Conexao conexao = new Conexao();
    Connection conn = conexao.conectar();
    try {
      String adicionar = "INSERT INTO PESSOA (NOME, IDADE) VALUES
('ROMULO', 24)";

      Statement stm = conn.createStatement();
      stm.execute(adicionar);
      System.out.println("Adicionou a pessoa Rafael no BD.");
    } catch (SQLException ex) {
      System.out.println("Não conseguiu adicionar uma pessoa no BD.");
    } finally {
      conexao.desconectar(conn);
    }
  }

  public void atualizar() {
    Conexao conexao = new Conexao();
    Connection conn = conexao.conectar();
    try {
      String atualizar = "UPDATE PESSOA SET NOME = 'Cristiano' WHERE
NOME = 'Rafael'";

      Statement stm = conn.createStatement();
      stm.executeUpdate(atualizar);
      System.out.println("Atualizou o nome de Rafael para Cristi-
ano.");
    } catch (SQLException ex) {
      System.out.println("Não conseguiu atualizar uma pessoa no BD.");
    } finally {
      conexao.desconectar(conn);
    }
  }
}
```

Na linha 24, pedimos para o `Statement` adicionar um novo registro na tabela `PESSOA` do banco de dados.

Na linha 40, pedimos para o `Statement` atualizar um registro da tabela `PESSOA` do banco de dados.

Note que para adicionar ou atualizar as informações no banco de dados, podemos utilizar o método `execute`.

A classe `Statement` possui o método execute para adicionar ou atualizar um registro no banco de dados e o método `executeUpdate` para atualizar as informações no banco de dados, a diferença é que este método retorna um inteiro com a quantidade de registros que foram alterados.

Após a conclusão das operações e leitura ou de manipulação de dados, é importante a chamada ao método `close()`, tanto da classe `Statement` como da classe `Connection`, para que a conexão com o banco de dados seja finalizada.

ATUALIZAÇÕES

Nesse ponto, já sabemos como consultar o banco de dados. Mas, e se quisermos alterar (*update*) alguma coisa por lá? “Tipo”, se inserirmos, atualizarmos ou removermos dados, estaremos alterando (*updating*) o estado do nosso banco. Faz sentido? Bom, para fazer isso, o código é “beeem” parecido com o anterior.

A mudança fica por conta do método que temos que chamar. Para executar a consulta (*query*), chamamos `executeQuery`. Para executarmos uma alteração (*update*), chamamos `executeUpdate`. Exemplo: se quisermos remover todos os “customers” que não estão ativos, escreveríamos o seguinte código:

```
// sua consulta
String sql = "delete from customer where active = ?";

// prepara a consulta, por meio da conexão (conn)
PreparedStatement stmt = conn.prepareStatement(sql);

// configura os parâmetros da consulta
stmt.setInt(1, 0); // colocar 0 na primeira interrogação

int n = stmt.executeUpdate();
```

Note que o `executeUpdate` nos retorna um inteiro. Esse número indica quantas linhas foram alteradas, inseridas ou removidas da tabela em questão. É interessante verificar sempre esse valor. Assim, podemos ter uma ideia se o comando foi ou não executado com sucesso. Exemplo: se não tiverem “customers” inativos, o `n` será `0`!

ENCERRANDO AS CONEXÕES

Como último passo da nossa “receita de bolo” para acesso a bancos de dados usando JDBC, é necessário finalizar, fechar a conexão com o banco de dados. Isso é importante porque, se não o fizermos, tanto o banco de dados como nosso programa passa a consumir recursos de máquina (memória, basicamente) de forma desnecessária.

Para fazer isso, a gente deve usar o método `close()` no objeto da classe `Connection`. No entanto, saiba que o `ResultSet` e o `PreparedStatement` também podem ser fechados. Ou seja, podemos deixar a conexão aberta, fechar apenas o conjunto de resultados, e reaproveitar a conexão aberta para fazer outra consulta. Aí, no final de tudo, deve-se fechar a conexão. Dessa forma, é importante seguir essa ordem no código.

```
rs.close();
stmt.close();
conn.close();
```

CONSIDERAÇÕES FINAIS

Apesar dos desenvolvedores contarem com frameworks JPA como Hibernate, que facilitam o acesso de aplicações Java a SGDB (Sistemas Gerenciadores de Banco de Dados), é bom saber o que acontece por trás dos bastidores desses frameworks.

Normalmente, isola-se a parte de acesso aos dados das telas da aplicação através de uma classe que recebe objetos de negócio e as persiste no banco de dados para posterior recuperação. Esse processo é o que chamamos de “mapeamento objeto-relacional”.

Vimos que o JDBC é uma API que pode acessar bancos de dados de forma independente. Também vimos que ele faz essa “mágica” por meio do uso dos conceitos de Programação Orientada a Objetos. Aliás, você notou que o único local do nosso código onde foi citado o MySQL foi nas configurações, mais precisamente dentro de uma string que poderia ter sido lida de um arquivo de configuração? Ou seja, nosso código mesmo nem sabe que está acessando o MySQL. Poderia ser o Oracle, o PostgreSQL ou qualquer outro!!!

REFERÊNCIAS

https://www.lncc.br/~rogerio/poo/09%20-%20Banco%20de%20Dados/java-bd-09b.pdf

http://www.universidadejava.com.br/java/java-jdbc/

https://nglauber.medium.com/acessando-banco-de-dados-mysql-com-jdbc-f0de021975b2

https://www.ramon.pro.br/jdbc-acessando-bancos-de-dados-em-java/

https://netbeans.apache.org//kb/docs/ide/java-db_pt_BR.html

http://www.linhadecodigo.com.br/artigo/1711/java-acesso-a-dados-usando-jdbc.aspx